Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
GRADUATION THESIS

Реализация интерфейса Django для работы с Yandex Database

Обучающийся / Student Зайнидинов Мирзофирдавс Шавкатович
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет информационных технологий и программирования
Группа/Group M34371
Направление подготовки/ Subject area 01.03.02 Прикладная математика и информатика
Образовательная программа / Educational program Информатика и программирование 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Руководитель ВКР/ Thesis supervisor Боркунов Владимир Юрьевич, Университет ИТМО, факультет информационных технологий и программирования, доцент (квалификационная категория "доцент практики")
Соруководитель не из ИТМО / Third-party co-supervisor Мясников Алексей Сергеевич, ООО Яндекс.Технологии, Руководитель команды разработки клиентских библиотек YDB, кандидат технических наук, старший научный сотрудник

Обучающийся/Student

Документ подписан
Зайнидинов Мирзофирдавс Шавкатович
17.05.2025

(эл. подпись/ signature)

Зайнидинов Мирзофирдавс Шавкатович
(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Боркунов Владимир Юрьевич
17.05.2025

(эл. подпись/ signature)

Боркунов Владимир Юрьевич
(Фамилия И.О./ name and surname)

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ / OBJECTIVES FOR A GRADUATION THESIS

Обучающийся / Student Зайнидинов Мирзофирдавс Шавкатович
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет информационных технологий и программирования
Группа/Group M34371
Направление подготовки/ Subject area 01.03.02 Прикладная математика и информатика
Образовательная программа / Educational program Информатика и программирование 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Реализация интерфейса Django для работы с Yandex Database
Руководитель ВКР/ Thesis supervisor Боркунов Владимир Юрьевич, Университет ИТМО, факультет информационных технологий и программирования, доцент (квалификационная категория "доцент практики")
Соруководитель не из ИТМО / Third-party co-supervisor Мясников Алексей Сергеевич, ООО Яндекс.Технологии, Руководитель команды разработки клиентских библиотек YDB, кандидат технических наук, старший научный сотрудник

Характеристика темы ВКР / Description of thesis subject (topic)

Название организации-партнера / Name of partner organization: ООО Яндекс.Технологии
Тема в области фундаментальных исследований / Subject of fundamental research: нет / not
Тема в области прикладных исследований / Subject of applied research: да / yes

Основные вопросы, подлежащие разработке / Key issues to be analyzed
Основные вопросы, подлежащие разработке

Целью данного проекта является Обеспечить полнофункциональное взаимодействие Django ORM с СУБД YDB, который:
Позволит использовать YDB как основную базу данных для Django-приложений.
Сохранит все преимущества Django ORM (производительность разработки, безопасность).
Обеспечит доступ к уникальным возможностям YDB.
Закроет существующий пробел в экосистеме Python/Django для работы с распределенными базами данных.

Основные задачи работы:

Анализ особенностей взаимодействия Django с СУБД.
Создание backend`а для YDB в Django.
Реализация всех обязательных классов/методов для Django.
Поддержка и настройки подключения к базе данных, методы для взаимодействия с клиентом базы данных.
Полная поддержка диалекта YQL, со всеми его особенностями:
Разработка транслятора YQL-запросов в ORM-модели и обратное преобразование.
Разработка методов для обработки уникальных типов данных YQL (например Optional, Yson, Dict).
Реализация методов для обработки специфичных операций YQL, (например оконные функции, метод UPSERT).
Реализация методов для автоматической конвертации между типами данных Django ORM и YQL.
Поддержка DDL запросов и миграций.
Проведение интеграционного и юнит тестирования.
Проведение нагрузочного тестирования для проверки производительности.
Подготовка документации и примеров использования для поддержки разработчиков.

Публикация результата в открытый репозиторий YDB.

Рекомендуемые материалы и пособия для выполнения работы:
Репозиторий Django: https://github.com/django/django.
Документация по YDB: https://ydb.tech/ru/.

Форма представления материалов ВКР / Format(s) of thesis materials:
Пояснительная записка, презентация, программный код.

Дата выдачи задания / Assignment issued on: 13.02.2025

Срок представления готовой ВКР / Deadline for final edition of the thesis 15.05.2025

СОГЛАСОВАНО / AGREED:

Руководитель ВКР/
Thesis supervisor

Документ подписан
Боркунов Владимир Юрьевич
17.05.2025

(эл. подпись)

Боркунов Владимир Юрьевич

Задание принял к исполнению/ Objectives assumed BY

Документ подписан
Зайнидинов Мирзофирдавс Шавкатович
17.05.2025

(эл. подпись)

Зайнидинов Мирзофирдавс Шавкатович

Руководитель ОП/ Head of educational program

Документ подписан
Станкевич Андрей Сергеевич
23.05.2025

(эл. подпись)

Станкевич Андрей Сергеевич

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

АННОТАЦИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ SUMMARY OF A GRADUATION THESIS

Обучающийся / Student Зайнидинов Мирзофирдавс Шавкатович
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет информационных технологий и программирования
Группа/Group M34371
Направление подготовки/ Subject area 01.03.02 Прикладная математика и информатика
Образовательная программа / Educational program Информатика и программирование 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Реализация интерфейса Django для работы с Yandex Database
Руководитель ВКР/ Thesis supervisor Боркунов Владимир Юрьевич, Университет ИТМО, факультет информационных технологий и программирования, доцент (квалификационная категория "доцент практики")
Соруководитель не из ИТМО / Third-party co-supervisor Мясников Алексей Сергеевич, ООО Яндекс.Технологии, Руководитель команды разработки клиентских библиотек YDB, кандидат технических наук, старший научный сотрудник

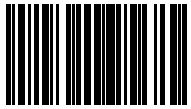
ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ DESCRIPTION OF THE GRADUATION THESIS

Цель исследования / Research goal
Реализовать backend, который обеспечить полнофункциональное взаимодействие Django ORM с СУБД YDB
Задачи, решаемые в ВКР / Research tasks
1. Реализовать все обязательные классы backend`а для полной совместимости Django ORM и СУБД YDB. 2. Разработать транслятор YQL-запросов в ORM-модели и обратное преобразование. 3. Покрыть реализацию юнит тестами и интеграционными тестами. 4. Провести нагрузочное тестирование для проверки производительности. 5. Проверить корректность реализации в готовом тестовом приложении Django, который использует СУБД YDB. 6. Подготовить документацию с примерами использования для поддержки разработчиков.
Краткая характеристика полученных результатов / Short summary of results/findings
Разработан backend, обеспечивающий полнофункциональное взаимодействие Django ORM с СУБД YDB, что позволяет создавать Django-приложения с использованием YDB. Проведено тестирование кодовой базы и нагрузочное тестирование. Также разработано тестовое Django-приложение для проверки реализации пользовательских сценариев. Подготовлена подробная документация с примерами использования для поддержки

разработчиков. По результатам сравнения производительности YDB и PostgreSQL в одинаковых условиях выявлено, что приложение на YDB демонстрирует достойную производительность и незначительно уступает PostgreSQL.

Обучающийся/Student

Документ подписан
Зайнидинов Мирзофирдавс Шавкатович
17.05.2025



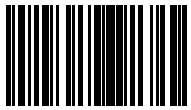
(эл. подпись/ signature)

Зайнидинов Мирзофирдавс Шавкатович

(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Боркунов Владимир Юрьевич
17.05.2025



(эл. подпись/ signature)

Боркунов Владимир Юрьевич

(Фамилия И.О./ name and surname)

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ....4
ВВЕДЕНИЕ....6
Глава 1. Обзор области и постановка задачи....10
1.1. Введение в YDB, Особенности и возможности YDB....10
1.2. Принципы и архитектура взаимодействия Django с базами данных....12
1.3. Постановка цели и задач....18
1.3.1. Требования к реализации....18
1.3.2. Цель....18
1.3.3. Задачи....19
ВЫВОДЫ ПО ГЛАВЕ 1....20
Глава 2. Предлагаемое решение....21
2.1. Обзор существующих реализаций....21
2.2. Архитектура решения....22
2.3. Реализация двустороннего транслятора Django ORM в YQL....25
2.4. DDL Запросы и миграции....30
ВЫВОДЫ ПО ГЛАВЕ 2....31
Глава 3. Тестирование....33
3.1. Подготовка тестовой инфраструктуры....33
3.1.1. Настройка линтеров для автоматических проверок кода....33
3.1.2. Настройка тестов для удобного запуска через простые команды....34
3.2. Интеграционное тестирование....39
3.3. Нагрузочное тестирование....42
ВЫВОДЫ ПО ГЛАВЕ 3....45
ЗАКЛЮЧЕНИЕ....46

БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....47

ВВЕДЕНИЕ

В современном мире информационные технологии играют ключевую роль во всех сферах деятельности человека. Эффективное управление данными и обеспечение их доступности является одной из важнейших задач для компаний и организаций. В связи с этим разработка и внедрение современных систем управления базами данных (СУБД) и инструментов для работы с ними становится приоритетным направлением.

Django [1] — это популярный фреймворк для разработки веб-приложений на языке Python, который предоставляет широкий спектр инструментов для быстрого создания качественных и надежных приложений. Одной из его сильных сторон является встроенная поддержка работы с различными СУБД через мощный инструмент — Django ORM [2] (Object-Relational Mapper).

Django — один из самых востребованных фреймворков для веб-разработки на Python, и его популярность обусловлена несколькими ключевыми факторами:

Универсальность и сферы применения. На Django разрабатывают широкий спектр приложений, включая:

- Корпоративные и высоконагруженные системы (CRM, ERP, системы документооборота).
- Социальные сети и платформы для контента (например, Instagram* изначально использовал Django).
- Электронную коммерцию (интернет-магазины, маркетплейсы) Финансовые и аналитические сервисы (банковские системы, финтех-приложения).
- Медиа Платформы и блоги (The Washington Times, Pinterest*).
- Образовательные и научные проекты (LMS, системы обработки исследовательских данных).

- API для мобильных приложений и микросервисов (сейчас эти компании используют кастомные решения, но начали с Django).

Разработчики и компании выбирают Django по нескольким ключевым причинам:

- Скорость разработки — Django предоставляет готовые решения для типовых задач: Аутентификация и авторизация. Админ-панель (Django Admin) ORM для работы с БД без написания SQL.
- Масштабируемость и надежность. Поддержка кэширования (Redis, Memcached).
- Безопасность — Django считается одним из самых безопасных фреймворков благодаря встроенной защите от уязвимостей. Автоматическому экранированию HTML/JS в шаблонах Поддержке HTTPS, сессий, ролевой модели доступа.
- Гибкость и экосистема. Поддержка PostgreSQL, MySQL, SQLite, Oracle из коробки. Возможность подключения NoSQL [3] (MongoDB, Elasticsearch) через сторонние библиотеки. Огромное количество пакетов.
- Поддержка и сообщество. Официальная документация Django считается одной из лучших. Большое количество русскоязычных материалов (актуально для СНГ).

Примеры известных проектов на Django:

- Disqus — система комментариев для сайтов.
- Mozilla Support — техподдержка Firefox.
- Spotify — внутренние инструменты анализа данных.
- YouTube — первоначальная версия (до перехода на Google инфраструктуру).

На момент января 2025 года Django являлся первым по популярности фреймворком для создания приложений. Статистика оценивалась по количеству звезд в репозитории GitHub. Статистика представлена на рисунке 1.

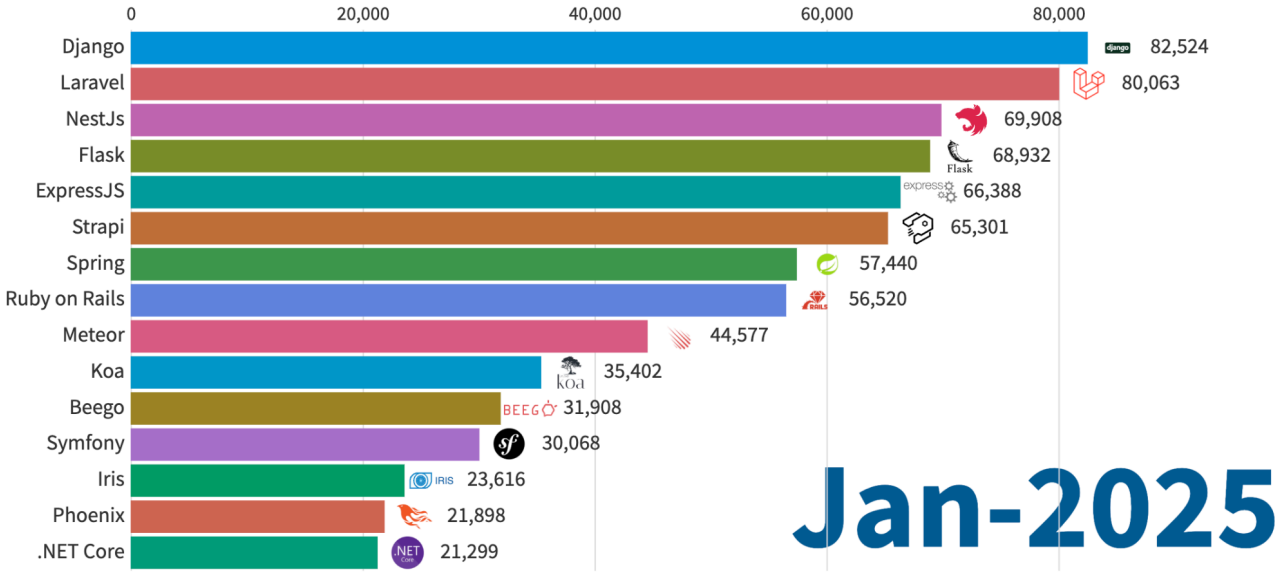


Рисунок 1 — Статистика популярных фреймворков для backend`а.

Django выбирают для проектов, где важны быстрота разработки, безопасность и долгосрочная поддержка.

YDB (Yandex Database) [4] — это распределённая СУБД. Она имеет открытый исходный код в Github и является хорошо масштабируемой. YDB имеет ряд преимуществ над другими СУБД, которое делает ее универсальным решением для различных задач.

Данная работа является очень актуальной в наши дни, так как облачные технологии стремительно развиваются и в связи с этим растет потребность в высокопроизводительных и хорошо масштабируемых СУБД, которые могут обрабатывать огромные объемы данных.

Yandex Database (YDB) является одним из ключевых продуктов экосистемы Yandex Cloud, предлагая уникальные возможности для работы с распределенными данными, включая автоматическое шардирование, высокую отказоустойчивость и низкие задержки при выполнении запросов. Однако, несмотря на растущую популярность YDB (по данным Yandex, количество активных пользователей сервиса увеличилось в 2,5 раза за 2022–2024 годы),

интеграция этой СУБД с одним из самых востребованных веб-фреймворков Django до сих пор отсутствует в виде готового решения.

Django остаётся одним из наиболее популярных инструментов для быстрой разработки веб-приложений. При этом Django традиционно ориентирован на реляционные базы данных (PostgreSQL, MySQL). Отсутствие официальной поддержки YDB в Django вынуждает разработчиков либо отказываться от использования этой СУБД, либо реализовывать не оптимальные обходные решения, что увеличивает затраты на разработку и снижает производительность конечных приложений.

Целью данной выпускной квалификационной работы является разработка и реализация интерфейса для работы с YDB в рамках фреймворка Django. Это позволит расширить возможности Django ORM и предоставить разработчикам удобный инструмент для работы с YDB. Данная работа устраняет этот пробел, предоставляя разработчикам удобный и стандартизированный способ работы с YDB в рамках Django-приложений, что способствует более широкому распространению как самого фреймворка, так и YDB в профессиональной среде. Таким образом, реализация интерфейса Django для YDB не только решает конкретную техническую задачу, но и способствует развитию экосистемы отечественных IT-решений, предоставляя разработчикам современный, эффективный и экономически выгодный инструмент для создания высоконагруженных приложений.

Для достижения поставленной цели необходимо решить следующие задачи:

- В главе 1 проводится обзор базы данных YDB и её особенностей, принципов в возможностей фреймворка Django, а также ставятся цели и задачи работы.
- В главе 2 рассматривается архитектура предложенного решения, описываются технические спецификации, особенности и трудности.

- В главе 3 представлено описание методов тестирования разработанной реализации поддержки YDB в Django. Освещается процесс нагрузочного тестирования, демонстрируется правильность решения с использованием стороннего тестового приложения и проводится анализ полученных результатов.

Результаты данной работы могут быть полезны разработчикам, работающим с Django и YDB, а также специалистам, интересующимся вопросами интеграции различных систем управления данными.

Глава 1. Обзор области и постановка задачи

1.1. Введение в YDB, Особенности и возможности YDB

YDB — это распределенная отказоустойчивая SQL СУБД. YDB обеспечивает высокую доступность, горизонтальную масштабируемость, а также строгую консистентность и поддержку ACID-транзакций [5]. Для запросов используется диалект SQL (YQL) [6]. В дизайн системы заложена работа с объемами данных в сотни петабайт. Для взаимодействия с YDB доступен YDB CLI, а также SDK для C++, C#, Go, Java, Node.js, PHP, Python и Rust [7]. YDB-SDK для языка Python реализует интерфейс YDB DB-API, который реализован по специфике pep 249 [8] В рамках данной работы будет улучшен опыт использования YDB на языке программирования Python.

YDB обладает следующими свойствами, о которых можно прочитать в документации YDB [4] по ключевым словам:

- Горизонтальная масштабируемость и отказоустойчивость. Автоматическое шардирование — данные распределяются по кластеру без участия разработчика.
- Высокая доступность (99.99% SLA) — репликация данных между зонами доступности.

- Поддержка кластеров из сотен узлов — может обрабатывать миллионы запросов в секунду.
- Полная поддержка ACID даже в распределенном режиме (в отличие от многих NoSQL-решений).
- Сквозные транзакции между таблицами и даже между разными базами данных.
- Бессерверный режим (Serverless) — не нужно управлять инфраструктурой [9].
- Совместимость с PostgreSQL — можно использовать клиентские библиотеки PG.
- Оптимизация для облачных и гибридных сред. Готовая интеграция с Yandex Cloud (Kubernetes, Message Queue, Object Storage).
- Поддержка гибридных развертываний — часть данных можно хранить локально.
- Открытый исходный код. Исходный код YDB опубликован под лицензией Apache 2 [10], накладываемой минимум ограничений на использование.

YDB используется в одних из крупнейших платформ и сервисах российского рынка, таких как Yandex Cloud, Яндекс Маркет, Auto.ru, Алиса.Основным средством создания, модификации и управления данными в YDB является декларативный, строго типизированный язык запросов YQL.

YQL (Yandex Query Language) — это специализированный декларативный и строго типизированный язык запросов, разработанный для работы с распределенными данными в YDB. В отличие от классического стандарта SQL [11], YQL адаптирован под особенности масштабируемых NoSQL/NewSQL-систем, что накладывает как уникальные возможности, так и определенные ограничения.

Ключевые особенности YQL:

- Поддерживает сложные типы данных, которые не имеют прямых аналогов в Django ORM:

- Реализация механизма контроля параллельного доступа. В YQL применяется стратегия оптимистического контроля версий (optimistic concurrency control), что накладывает определенные ограничения по сравнению с традиционными СУБД:
    - Отсутствует поддержка пессимистических блокировок (pessimistic locking).
    - Конфликты параллельных транзакций выявляются только на этапе фиксации изменений.
    - При возникновении коллизий система автоматически отменяет одну из конкурирующих транзакций.
- Особенности работы с первичными ключами. YQL предъявляет специфические требования к генерации идентификаторов:
    - Автоматическая генерация ключей (auto-increment) не поддерживается.
    - Значения первичных ключей должны быть явно заданы клиентом перед выполнением запроса.
- Не поддерживаются некоторые конструкции, имеющиеся в других диалектах SQL и наоборот поддерживает некоторые конструкции, не имеющиеся в других диалектах SQL.

1.2. Принципы и архитектура взаимодействия Django с базами данных

Django ORM (Object-Relational Mapping) представляет собой мощный и элегантный способ взаимодействия с реляционными базами данных, реализующий концепцию "активной записи" (Active Record). Django ORM разработан для упрощения работы с данными. Он является очень удобным инструментом при этом не теряет гибкость и производительность.

Можно перечислить несколько основных принципов Django ORM:

- Простота и продуктивность. Django ORM описывает сущности как Python объекты и автоматически генерирует соответствующие для них

SQL-запросы. Это значительно сокращает объем написанного кода необходимого для реализации разных операций над данным, такие как CRUD [12] операции, и соответственно ускоряет разработку, при этом не теряет преимущества реляционных СУБД.

- Доменно-ориентированный подход. Каждая модель Django представляет собой отдельную сущность с определенными полями и связями между сущностями. Django ORM обрабатывает преобразование между этими сущностями учитывая связи между сущностями. Это позволяет разработчику сосредоточиться на бизнес логике разрабатываемого приложения.
- Полноценная взаимодействие с другими компонентами Django. Django ORM тесно взаимодействует с другими компонентами, которые есть в Django. Эти компоненты включают в себя:
    - Систему работы с аутентификации.
    - Административный интерфейс.
    - Механизм работы с формами.

  Эти взаимодействия обеспечивают согласованную работу всех частей приложения без перебоев.

Рассмотрим архитектуру Django ORM для взаимодействия с СУБД:

- Модели (Models) — сущности в Django ORM, которые представляют таблицы в виде Python-классов. Созданные модели наследуются от класса Models, который находится пакете django.db.models. Модели определяют структуру таблиц базы данных и включают в себя:
    - Определение полей таблицы.
    - Определение связей таблиц.
    - Определение разных метаданных, таких как имя таблицы для работы с ней, краткое описание, строковое представление атрибутов таблицы и другие.

- Менеджеры моделей и QuerySets. В Django ORM менеджеры моделей представляют собой интерфейс для работы с базой данных. Они позволяют выполнять различные операции над таблицей, такие как создать запись, изменить запись, удалить запись. QuerySets это по сути “ленивая” коллекция объектов. Он не выполняет запросы к базе сразу, а только тогда, когда это действительно нужно. С помощью QuerySets можно эффективно выполнять операции фильтрации, агрегации и другие.
- Система миграций в Django ORM. С помощью системы миграций в случаи изменений в Django моделях можно её запустить, после чего генерируется и запускаются соответствующие SQL-запросы для обновления и актуализации схемы базы данных. Система миграций также поддерживает историю изменений и возможность отката, чтобы разработчик в случае чего мог откатить свои изменения.
- Поддержка транзакций в Django ORM. Django предоставляет несколько уровней управления транзакциями: Автоматическая обработка на уровне HTTP-запросов Явное управление через transaction.atomic Поддержка точек сохранения (savepoints)
- Возможность одновременной работы с несколькими базами данных через механизм маршрутизации запросов и явное указание используемой базы для каждой операции.
- Адаптеры баз данных Django поддерживает различные СУБД через систему backend`ов, включая:
    - PostgreSQL
    - MySQL/MariaDB
    - SQLite
    - Oracle
    - Возможность создания кастомных backend`ов

Django ORM находит идеальный баланс между производительностью и удобством, делая его отличным выбором для широкого круга веб-приложений -

от небольших проектов до высоконагруженных систем. Его архитектура позволяет эффективно работать с данными, минимизируя при этом количество потенциальных ошибок и время разработки.

Пример простой модели на Django можно посмотреть на листинге 1.

```
from django.db import models
from django.contrib.auth import get_user_model

# Получаем модель пользователя
User = get_user_model()

class Article(models.Model):
    title = models.CharField(
        max_length=200,
        verbose_name="Заголовок"  # Человеко читаемое имя
    )
    content = models.TextField(
        verbose_name="Содержание"
    )
    author = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name='articles',
        verbose_name="Автор"
    )
    published_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Дата публикации"
    )
    views = models.PositiveIntegerField(
        default=0,
        verbose_name="Просмотры"
    )
    tags = models.ManyToManyField(
        'Tag',  # Ссылка на модель в том же файле
        blank=True,
        related_name='articles',
        verbose_name="Теги"
    )
```

```
    class Meta:
        # Мета-класс для дополнительных настроек модели.
        ordering = ['-published_at']
        verbose_name = 'Статья'

    def __str__(self):
        # Метод для строкового представления объекта.
        return self.title

class Tag(models.Model):
    name = models.CharField(
        max_length=50,
        unique=True,
        verbose_name="Название тега"
    )

    def __str__(self):
        return self.name
```

Листинг 1 — пример просто модели на Django.

На рисунке 2 представлена Django модель из листинга 1 в виде модели сущность-связь.

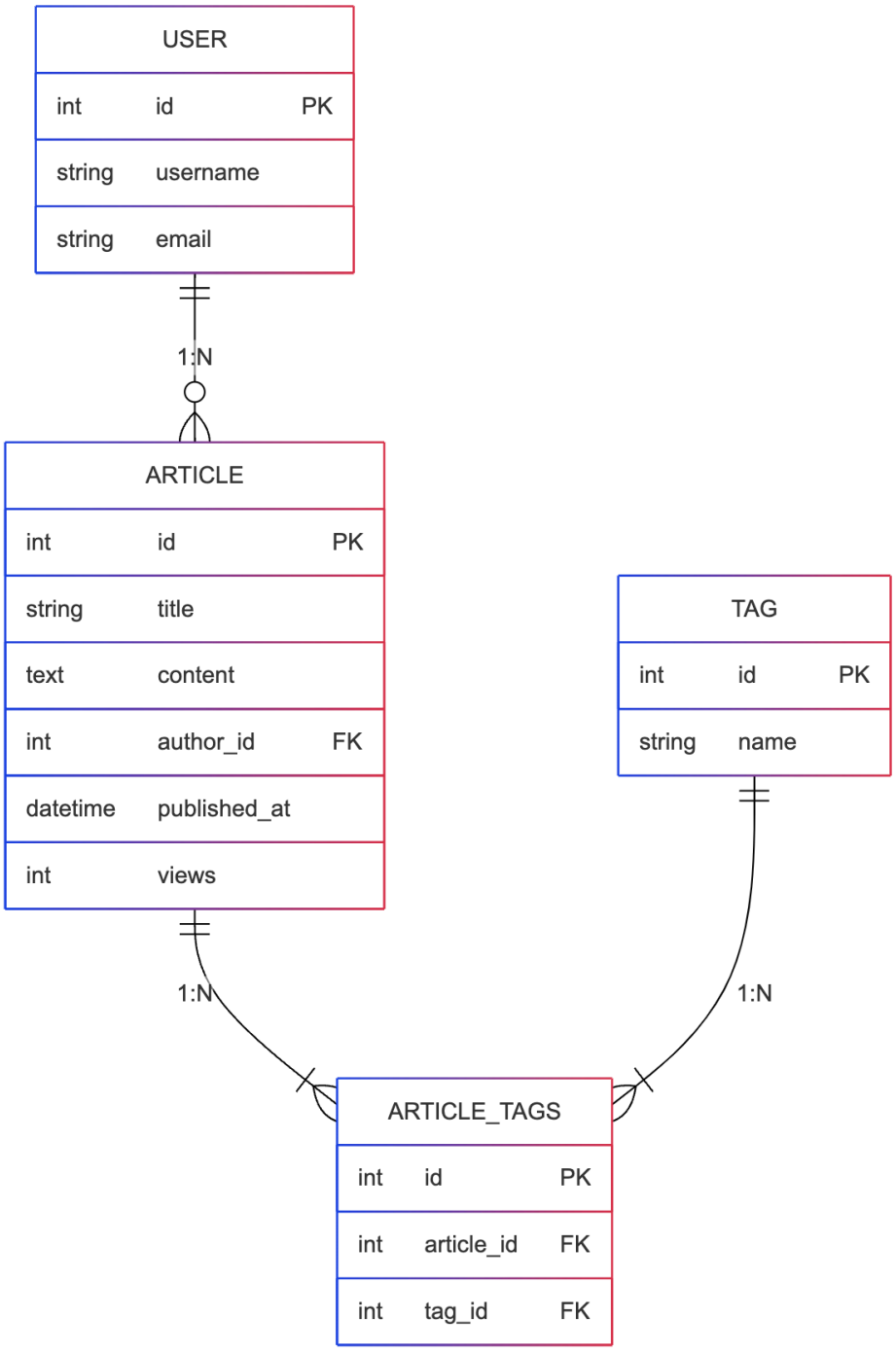


Рисунок 2 — Модель сущность-связь для Django модели из листинга 1.

1.3. Постановка цели и задач

YDB предоставляет официальный Python SDK для работы с базой данных. На основе этого SDK необходимо реализовать полноценную поддержку Django ORM, создав адаптер для интеграции YDB в экосистему Django.

1.3.1. Требования к реализации

- Поддержка базового полнофункциональный ORM:
    - Поддержка моделей (Models).
    - CRUD-операции.
    - Запросы и агрегации.
    - Транзакции и атомарность.
    - Миграции.
    - Индексы и ограничения.
    - Поддержка поиска (search_fields), фильтрации (list_filter).
    - Валидация и сохранение данных.
    - Поддержка Raw SQL и кастомных запросов Выполнение сырых SQL-запросов через .raw() и cursor.execute().
    - Оптимизация производительности.
    - Кэширование часто используемых запросов.
- Корректное сопоставление типов между Python и YDB.
- Поддержка уникальных типов и методов YDB.

1.3.2. Цель

Разработать первый полнофункциональный Django backend для YDB, который:

- Позволит использовать YDB как основную базу данных для Django-приложений.
- Сохранит все преимущества Django ORM (производительность разработки, безопасность).
- Обеспечит доступ к уникальным возможностям YDB.

- Закроет существующий пробел в экосистеме Python/Django для работы с распределенными базами данных.

1.3.3. Задачи

- Анализ особенностей взаимодействия Django с СУБД.
- Создание backend`a для YDB в Django:
    - Реализация всех обязательных классов/методов для Django.
    - Поддержка и настройки подключения к базе данных, методы для взаимодействия с клиентом базы данных.
    - Полная поддержка диалекта YQL, со всеми его особенностями:
        - Разработка транслятора YQL-запросов в ORM-модели и обратное преобразование.
        - Разработка методов для обработки уникальных типов данных YQL (например Optional, Yson, Dict).
        - Реализация методов для обработки специфичных операций YQL, (например оконные функции, метод UPSERT).
        - Реализация методов для автоматической конвертации между типами данных Django ORM и YQL.
    - Поддержка DDL [13] запросов и миграций.
- Проведение интеграционного тестирования и юнит тестирования.
- Проведение нагрузочного тестирования для проверки производительности.
- Подготовка документации и примеров использования для поддержки разработчиков
- Публикация результата в открытый репозиторий YDB.

ВЫВОДЫ ПО ГЛАВЕ 1

В данной главе проведен обзор возможностей и особенностей YDB и Django ORM. Определены ограничения технологий и требования к реализации. Четко изложены цель и задачи выпускная квалификационная работы.

Глава 2. Предлагаемое решение

2.1. Обзор существующих реализаций

На данный момент имеется единственный репозиторий django-connector [14], в котором предпринималась попытка решения этой задачи, однако разработка была прекращена на ранних этапах из-за недостатка ресурсов и объема предстоящей работы. В этом решении присутствуют ряд проблем, некоторые из которых это:

- Неполная реализация методов:
    - Отсутствует большая часть методов.
    - Некорректно реализованы существующие методы (например, в проекте используются такие методы как: SHOW TABLES, DESCRIBE и другие, которые не поддерживаются диалектом YQL).
- Проблемы совместимости:
    - Отсутствует преобразование диалекта YQL в Django ORM.
    - Не реализованы обязательные базовые классы (например, класс Operations, который содержит методы для выполнения различных операций с данными, таких как сравнение, арифметические операции и т. д.).
- Проблемы конфигурации:
    - Неправильные параметры подключения (отсутствуют host, port и другие обязательные параметры).
    - Полностью отсутствует тестовая инфраструктура.
- Критические ошибки:
    - Код не проходит даже этап подключения к базе данных.
    - Нулевое тестовое покрытие.

В связи с этим, представленное решение не решает данную проблему от слово совсем.

2.2. Архитектура решения

Архитектурная схема взаимодействия Django ORM с YDB предоставлен на рисунке 3.

Пошаговое объяснение работы:

- Первый шаг. Django ORM преобразует методы Python такие как filter(), save(), annotate() и другие в абстрактные запросы.
- Второй шаг, это шаг, который нам нужно реализовать. Django YDB Backend определяет как нужно транслировать стандартные операции из Django ORM в YQL диалект, учитывая все специфики диалекта.
- Шаг третий. Сгенерированный YQL запрос передаётся в официальный SDK YDB через DB-API-интерфейс.
- Шаг четвертый. Запрос отправляется в кластер YDB через gRPC протокол.

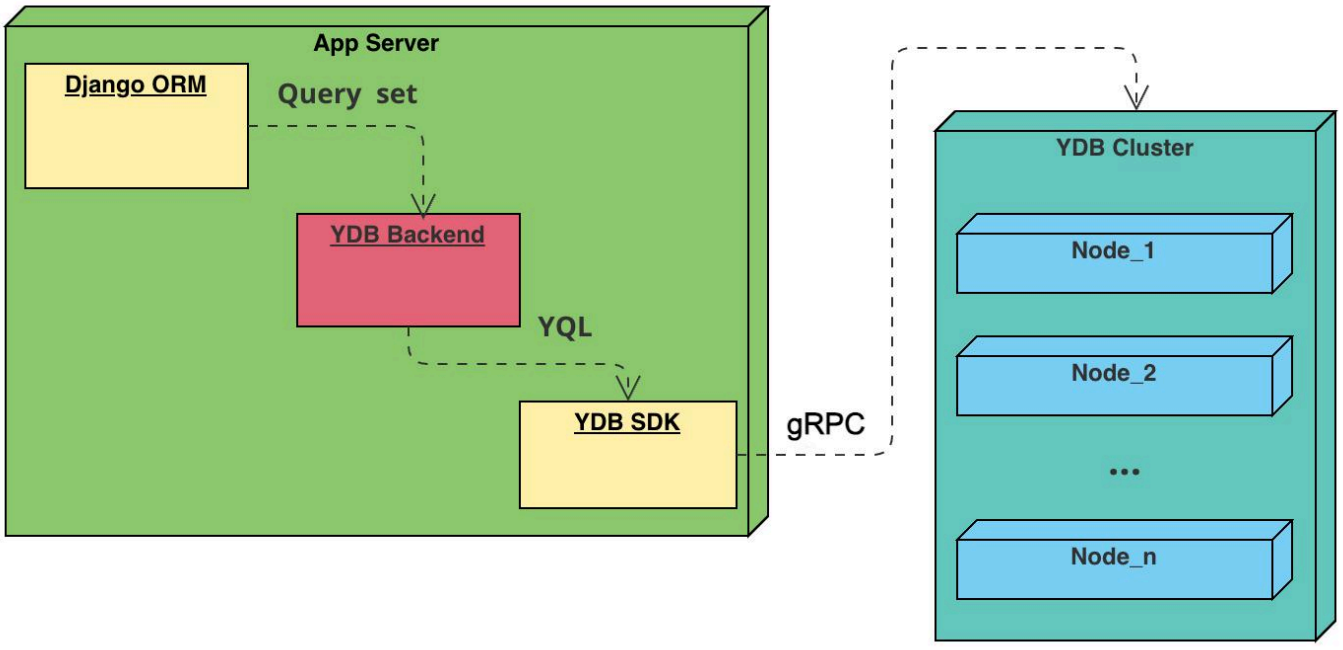


Рисунок 3 — Архитектурная схема взаимодействия Django ORM с YDB в стандартной UML нотации.

Диаграмму классов разработанного решения можно посмотреть на рисунке 4.
Django есть специальный интерфейс, для работы с базами данных и каждый реализованный Django backend, реализует методы этих самых интерфейсов. В решении задачи присутствуют 6 обязательных классов и один опциональный, которые описывают весь функционал и работу Django backend`a. Каждый класс в Django backend выполняет строго определённую роль в работе Django ORM с YDB.
Ниже рассмотрены детальное описание этих классов, какие функции они выполняют и как они взаимодействуют между собой и другими компонентами Django ORM:

- Класс DatabaseWrapper – главный класс, который выступает мостом между Django ORM и YDB. DatabaseWrapper наследуется от базового класса BaseDatabaseWrapper, который реализован в Django. DatabaseWrapper переопределяет и расширяет возможности базового класса, для работы с YDB. DatabaseWrapper управляет подключением к базе данных, управлением транзакций и выполняет YQL-запросы.
- Класс DatabaseFeatures отвечает за поддержку разных возможностей базы данных YDB. DatabaseFeatures наследуется от базового класса BaseDatabaseFeatures, который реализован в Django. DatabaseFeatures содержит булевые флаги, которые определяют какие возможности поддерживает СУБД. Далее эти флаги используется во всех классах backend`a, и в зависимости от значений этих флагов выполняется или не выполняется та или иная логика в коде.

  Пример флагов:
  - Поддерживает ли СУБД внешние ключи.
  - Умеет ли СУБД переименовывать индексы.
- Класс DatabaseCreation наследуется от базового класса BaseDataBaseCreation и реализует методы управления жизненного цикла

тестовой базы данных YDB (создание базы данных, удаление базы данных, клонирования и так далее)

- Класс DatabaseClient наследуется от базового класса BaseDatabaseClient и реализует методы взаимодействия с клиентом базы данных. С помощью этого класса, можно преобразовывать настройки Django в аргументы командной строки, также с помощью метода run_shell() можно запуска запускать интерактивную консоль YDB.
- Класс DatabaseShemaEditor наследуется от базового класса BaseDatabaseEditor и реализует методы для операций над схемой базы данных (DDL операции), таких как создание таблиц и индексов, их редактирование и удаление и так далее. Этот класс является ядром системы миграций Django.
- Опциональный класс DatabaseValidation наследуется от базового класса BaseDatabaseValidation и реализует методы для проверки совместимости полей Django моделей с особенностями YDB. Например поддерживает ли YDB конкретные типы поля в сочетаниями с определенным атрибутом (например индексы) и другие такого рода проверки.
- Класс DatabaseIntrospection отвечает за метаданные таблиц, которые есть в базе данных. DatabaseIntrospection наследуется от базового класса BaseDatabaseIntrospection, который реализован в Django. В классе DatabaseIntrospection реализованы следующие методы:
    - Возвращение списка имен существующих таблиц в базе данных.
    - Возвращение атрибутного состава, по названию таблицы.
    - Возвращения информации о связей таблиц.
    - Возвращение описание таблицы по названию.
    - Возвращение списка ограничений по названию таблицы.
    - И другие методы.
- Класс DatabaseOperations является ключевым классом для формирования YQL-запросов. DatabaseOperations наследуется от базового класса

BaseDatabaseOperations, который находится в DatabaseOperations реализует методы для преобразования операций Django ORM в YQL-запросы. На вход он получает операцию, которая выполняется для модели, а на выходе он возвращает YQL-запрос этой операции.

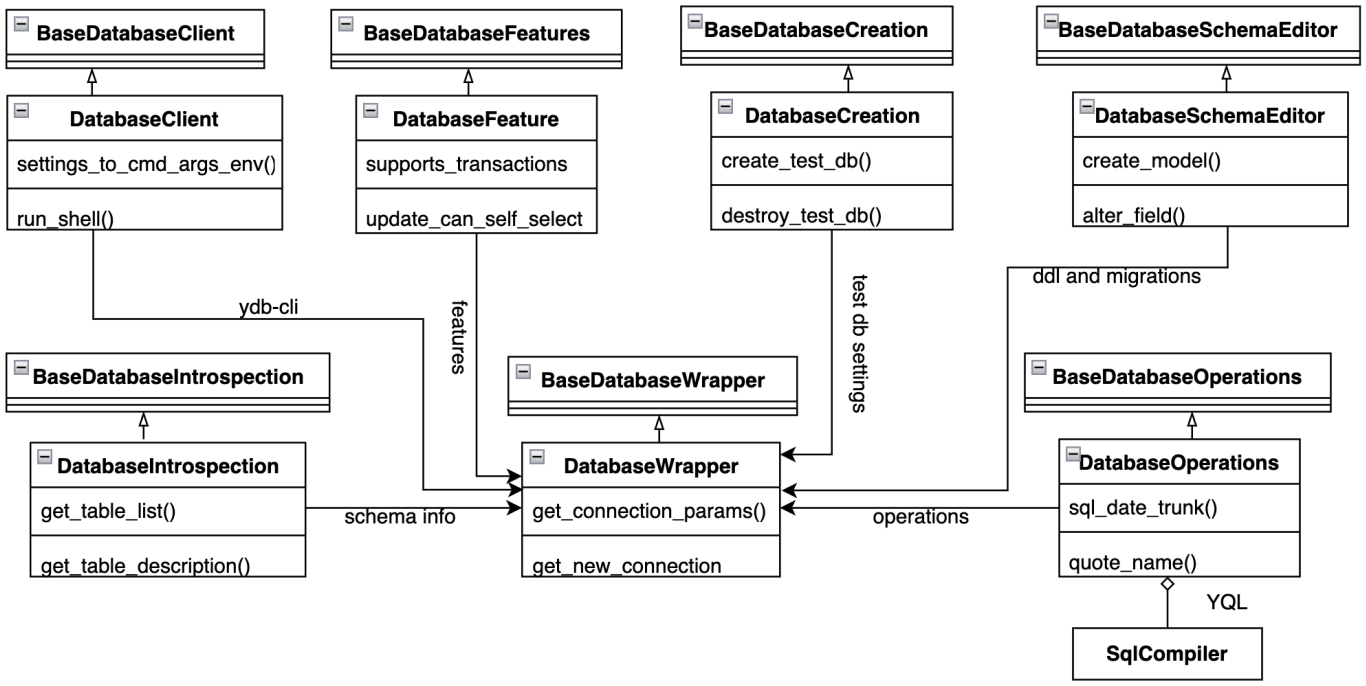


Рисунок 4 — Диаграмма классов backend`а и их взаимодействие друг с другом в UML нотации.

2.3. Реализация двустороннего транслятора Django ORM в YQL

Транслятор из Django ORM в YQL, реализован в файле compiler.py. Система компиляции SQL в Django состоит из нескольких ключевых классов, которые преобразуют высокоуровневые QuerySet в конкретные YQL-запросы. Основные компоненты:

SQLCompiler (базовый)

- ├── SQLInsertCompiler
- ├── SQLUpdateCompiler
- ├── SQLDeleteCompiler
- ├── SQLUpsertCompiler

└── SQLAggregateCompiler

Класс SQLCompiler является ядром системы преобразования запросов в Django ORM и выполняет важные функции для преобразования высокоуровневых объектов QuerySet в конкретные YQL-запросы. Это сложный механизм, такой как:

- Абстрагирует различия между реляционными базами данных.
- Правильно укажите параметры для обеспечения безопасности.
- Реализует оптимизацию для создания эффективного YQL

Компилятор выступает в качестве промежуточного звена между абстрактным представлением запроса (модель, условия фильтрации, аннотации) и конкретной реализацией YQL в целевой базе данных.

SQLCompiler занимает центральное положение в архитектуре Django ORM, взаимодействуя с множеством других компонентов:

- QuerySet - получает "сырой" не скомпилированный запрос.
- DatabaseOperations - использует для YDB-специфичных операций.
- Выражения (Expressions) - рекурсивно компилирует сложные условия.
- Исполнитель запросов - возвращает готовый YQL для выполнения Этот класс реализует паттерн "Компилятор" (Compiler pattern), где сложный объект (Query) преобразуется в другой формат представления (YQL).

Основная работа происходит в несколько этапов:

- Анализ структуры запроса (определение таблиц, полей, условий).
- Рекурсивная компиляция выражений (WHERE, HAVING, ORDER BY).
- Генерация YQL-фрагментов (SELECT, FROM, JOIN).
- Сборка итогового запроса с учетом всех компонентов.

Примеры основных методов:

- as_sql() - главный метод, который собирает sql запрос и список его параметров
- execute_sql() - выполняет сгенерированный запрос с помощью метода курсора из YDB SDK.

Метод execute_sql() для SELECT-запросов (result_type != None) — использует курсор и выполняет cursor.execute(sql, params). Для INSERT/UPDATE/DELETE/UPSERT (методы которых реализуют классы SQLInsertCompiler, SQLUpdateCompiler, SQLDeleteCompiler, SQLUpsertCompiler) (result_type=None) — выполняет запрос и возвращает количество изменённых строк.

Классы SQLInsertCompiler, SQLUpdateCompiler, SQLDeleteCompiler, SQLUpsertCompiler также наследуются от класса SQLCompiler, и реализуют два метода as_sql() и sql_execute().

Класс SQLAggregateCompiler – это специализированный компилятор YQL, который обрабатывает агрегатные запросы (например, annotate(), aggregate()). Он наследуется от базового SQLCompiler и переопределяет метод as_sql(), чтобы генерировать YQL запросы для агрегаций.

SQLAggregateCompiler выполняет задачи:

- Обработки агрегатных функций. Он преобразует вызовы вроде Count(), Sum(), Avg() в YQL-выражения (COUNT(*), SUM(column), и т. д.).
- Генерация подзапросов. Агрегации часто требуют вложенных запросов.
- Работа с annotate() и aggregate().

Взаимодействие с DatabaseOperations строится на нескольких ключевых аспектах:

- Делегирование СУБД-специфичных операций.
    - Все особенности синтаксиса конкретной базы данных выносятся в ops.
    - Компилятор запрашивает нужные YQL-шаблоны при генерации.
- Единые точки интеграции.
    - Форматирование идентификаторов (quote_name).
    - Специальные функции (date_extract_sql).
    - Обработка выражений (combine_expression).
- Согласованность интерфейсов.

    - Четкие контракты между компилятором и ops.
    - Стандартизированные форматы возвращаемых значений.

Сложными аспектами реализации являются:

- Рекурсивная компиляция выражений. Система построена на глубокой рекурсии при обработке:
    - Вложенных условий WHERE.
    - Комплексных аннотаций.
    - Подзапросов и оконных функций.

Каждый уровень выражения может требовать обращения к DatabaseOperations, для специфичных конструкций.

- Управление алиасами и областями видимости. Сложный механизм отслеживания:
    - Временных имен таблиц.
    - Областей видимости в подзапросах.
    - Конфликтов имен при соединениях
- Оптимизация структуры запроса. Компилятор выполняет:
    - Упрощение логических выражений.
    - Устранение избыточных условий.
    - Оптимизацию порядка соединений.

Как пример оптимизации структуры запроса, можно рассмотреть поддержку параметризованных запросов [15] в insert/upsert запросах. YDB поддерживает и рекомендует к использованию так называемые параметризованные запросы. В таких запросах данные передаются отдельно от самого тела запроса, а в YQL-запросе используются специальные параметры для обозначения местоположения данных.

Запрос с данными в теле запроса можно посмотреть на листинге 3.

```
SELECT sa.title AS season_title, sr.title AS series_title
FROM seasons AS sa INNER JOIN series AS sr ON sa.series_id = sr.series_id
WHERE sa.series_id = 15 AND sa.season_id = 3
```

Листинг 3 — Запрос с данными в теле запроса.

Соответствующий ему параметризованный запрос можно посмотреть на листинге 4.

```
DECLARE $seriesId AS Uint64;
DECLARE $seasonId AS Uint64;

SELECT sa.title AS season_title, sr.title AS series_title
FROM seasons AS sa INNER JOIN series AS sr ON sa.series_id = sr.series_id
WHERE sa.series_id = $seriesId AND sa.season_id = $seasonId
```

Листинг 4 — Параметризованный запрос.

Параметризованные запросы представляют собой специальный формат записи SQL-выражений, в котором вместо конкретных значений используются именованные или позиционные псевдопеременные. В синтаксисе YQL эти параметры традиционно обозначаются через символ доллара с последующим идентификатором (например, $seriesId, $seasonId), которые затем заменяются фактическими значениями при выполнении запроса.

Преимущества таких запросов:

- Оптимизация выполнения повторяющихся запросов. Система управления базами данных обладает способностью сохранять в кэше сформированные планы выполнения для параметризованных запросов. Такой подход существенно уменьшает нагрузку на центральный процессор и значительно увеличивает общую производительность системы при массовой обработке однотипных запросов.
- Защита от внедрения SQL-кода (SQL Injection). Применение параметризованных запросов полностью исключает возможность интерпретации внешних данных как исполняемого SQL-кода, тем самым устраняя один из наиболее опасных видов уязвимостей в веб-приложениях.
- Автоматическое кэширование планов запросов в YDB. В рамках YDB SDK предусмотрена встроенная система кэширования планов

выполнения для параметризованных запросов. Данная функциональность активируется по умолчанию через параметр KeepInCache = true, что обеспечивает оптимальную производительность без необходимости дополнительной настройки со стороны разработчика.

- Унификация и стандартизация запросов. Использование параметризованных запросов помогает привести структуру запросов к единообразию. Этот подход должен упрощать анализ и отладку запроса.

2.4. DDL Запросы и миграции

Как упоминалось выше, в Django работа DDL-запросами (Data Definition Language) реализовано в классе BaseDatabaseSchemaEditor. BaseDatabaseSchemaEditor представляет базовую реализацию для работы с DDL запросами и миграциями. В нашей реализации класс BaseDatabaseSchemaEditor расширен с помощью класса DatabaseSchemaEditor, в котором переопределены, некоторые методы, которые отличаются в YDB от стандартной реализации и добавлены новые методы, которые отсутствуют в стандартной реализации.
Ключевые аспекты реализации класса DatabaseSchemaEditor:

- Генерация YQL-специфичного DDL:
    - Переопределение базовых методов Django для соответствия синтаксису YQL.
    - Особенности обработки первичных ключей (в YDB они обязательны).
    - Специфическая реализация создания/изменения индексов (создание изменения и типы индексов отличаются от стандарта SQL).
- Адаптация миграционных операций:
    - Трансформация стандартных миграций Django в YDB-совместимый формат.
    - Обработка ограничений YDB (например, невозможность ALTER COLUMN, или же отсутствия CONSTRAINT).

    - Эмуляция неподдерживаемых операций через последовательность допустимых (например создание индекса через его добавление в таблицу через ALTER).

Реализация DDL-запросов и миграций в YDB Backend демонстрирует гибкость архитектуры Django, позволяющей адаптировать систему управления схемой данных под особенности YDB. Класс DatabaseSchemaEditor служит важным связующим звеном между стандартными миграциями Django и специфичной реализацией YDB, обеспечивая:

- Корректное выполнение операций со схемой.
- Сохранение совместимости с инструментарием Django.
- Эффективную работу с особенностями YDB.
- Безопасное применение изменений.

Представленная решение учитывает как текущие ограничения YDB, так и возможности для дальнейшего расширения функциональности по мере развития самой YDB.

ВЫВОДЫ ПО ГЛАВЕ 2

Проведен обзор существующих реализаций и их недостатки по сравнения с моим решением. Разработанное решение обеспечивает полную интеграцию Django ORM с YDB через систему специализированных классов-адаптеров. Ключевые особенности реализации:

- Многоуровневая архитектура трансляции ORM-запросов в YQL с сохранением стандартного интерфейса Django.
- Поддержка специфичных типов и методов диалекта YQL.
- Полноценная поддержка DDL-операций и миграций с учетом специфики YDB.
- Оптимизированная система компиляции запросов с использованием параметризации.
- Соответствие стандартам безопасности и производительности Django.

Решение демонстрирует эффективный подход к адаптации ORM, сохраняя все преимущества фреймворка при работе с YDB.

Глава 3. Тестирование

3.1. Подготовка тестовой инфраструктуры

В рамках этой задачи подготовлена тестовая инфраструктура, чтобы было удобнее и легче запускать тесты через простые команды. Также настроены линтеры для автоматического анализа кода. Решение упаковано с помощью Poetry [16] — современного инструмента для управления зависимостями и сборки.

3.1.1. Настройка линтеров для автоматических проверок кода

Линтеры — это инструменты для автоматического анализа кода, которые:

- Находят ошибки (синтаксис, типы, потенциальные баги).
- Проверяют стиль (PEP 8, именование, отступы).
- Обеспечивают единые стандарты в команде.
- Экономят время на ручной review.

В проекте используется Ruff [17] — современный высокопроизводительный инструмент для статического анализа Python-кода, объединяющий функциональность:

- flake8 (стиль кода).
- isort (сортировка импортов).
- pylint (расширенный анализ).
- и других линтеров.

Линтер добавлен как dev-зависимость в pyproject.toml. Код можно посмотреть на листинге 5.

```
[tool.poetry.group.dev.dependencies]
ruff = "^0.9.9"
```

Листинг 5 — Код на добавления линтера как dev-зависимость.

Настройки линтера можно посмотреть на листинге 6.

```
[tool.ruff]
exclude = [".venv", ".git", ".github/scripts", "__pycache__", "build",
"dist", "venv"] # Игнорируемые директории
line-length = 88 # Максимальная длина строки
target-version = "py39" # Целевая версия Python (3.9+)
src = ["ydb_backend", "tests"] # Директории для проверки
```

Листинг 6 — Настройки линтера.

Ruff имеет много полезных комманд:

- poetry run ruff check — Запуск проверок без исправлений.
- poetry run ruff check --fix — Запуск проверок с исправлениями.
- poetry run ruff format . — Форматировать код.
- и другие.

3.1.2. Настройка тестов для удобного запуска через простые команды

Есть docker-compose.yml файл, который:

- Разворачивает локальный YDB-сервер для тестов.
- Данные хранятся в оперативной памяти (чтобы тесты работали быстро).
- Пробрасывает порты:
    - 2136 — для подключения Django (через ydb_backend).
    - 8765 — веб-интерфейс YDB (можно открыть в браузере).

Более подробно можно наблюдать на листинге 7.

```
version: "3.3"
services:
 ydb:
   image: ydbplatform/local-ydb:trunk
   restart: always
   ports:
     - 2136:2136
     - 8765:8765
   hostname: localhost
   environment:
     - YDB_USE_IN_MEMORY_PDISKS=true
```

```
      - YDB_ENABLE_COLUMN_TABLES=true
```

Листинг 7 — Скрипт docker-compose.yml.

Далее есть файлы settings.py и runntests.py.

В settings.py описываются параметры для подключения к тестовой базе YDB.

Более подробно можно посмотреть на листинге 8.

```
from pathlib import Path

BASE_DIR = Path(__file__).parent


DATABASES = {
   "default": {
       "NAME": "ydb_db",
       "ENGINE": "ydb_backend.backend",
       "HOST": "localhost",
       "PORT": "2136",
       "DATABASE": "/local",
       "OPTIONS": {
           "credentials": None,
       },
   }
}
```

Листинг 8 — Файл settings.py.

runtests.py представляет собой кастомный запускатель тестов для Django, адаптированный для тестирования YDB backend`a. Разберем его функциональность.

Скрипт выполняет следующие задачи:

- Сканирует директорию tests/ и находит все тестовые модули.
- Настраивает Django для работы с YDB через settings.py.
- Запускает тесты с поддержкой различных опций (уровень детализации, сохранение БД и т.д.).
- Возвращает статус выполнения (0 — успех, 1 — ошибка).

Ключевые компоненты:

- Метод поиска тестовых модулей, можно посмотреть на листинге 9.

```
def get_test_modules():
    """
    Scan the tests directory and yield the names of all test modules.
    """
    with os.scandir(RUNTESTS_DIR) as entries:
        for f in entries:
            if (
                "." in f.name
                or Path(f.name).name in SKIP_DIRS
                or f.is_file()
                or not (Path(f.path) / "__init__.py").exists()
            ):
                continue
            test_module = f.name
            yield test_module
```

Листинг 9 — Метод поиска тестовых модулей.

Метод рекурсивно ищет папки в tests/, содержащие __init__.py. Игнорирует скрытые файлы. Папки из списка SKIP_DIRS. Обычные файлы (не-пакеты).

- Парсинг аргументов командной строки python можно посмотреть на листинге 10.

```
parser = argparse.ArgumentParser(
    description="Run the Django test suite for YDB backend."
)
parser.add_argument(
    "modules",
    nargs="*",
    metavar="module",
    help='Optional path(s) to test modules; e.g. "tests.test_basic".',
)
parser.add_argument(
    "-v",
    "--verbosity",
    default=1,
    type=int,
    choices=[0, 1, 2, 3],
```

```
        help="Verbosity level; 0=minimal output, 1=normal output, 2=all output",
    )
    parser.add_argument(
        "--noinput",
        action="store_false",
        dest="interactive",
        help="Tells Django to NOT prompt the user for input of any kind.",
    )
    parser.add_argument(
        "--failfast",
        action="store_true",
        help="Tells Django to stop running the test suite after the first failed
    test.",
    )
    parser.add_argument(
        "--keepdb",
        action="store_true",
        help="Tells Django to preserve the test database between runs.",
    )
    parser.add_argument(
        "--debug-sql",
        action="store_true",
        help="Turn on the SQL query logger within tests.",
    )

    options = parser.parse_args()
    options.modules = [os.path.normpath(labels) for labels in options.modules]
```

Листинг 10 — Код парсинга аргументов командной строки.

- Поддерживаемые флаги:
    - modules — список тестов для запуска (если не указан, тестируются все).
    - --verbosity — уровень детализации вывода (0 — минимум, 3 — максимум).
    - --noinput — отключает интерактивные запросы (например, подтверждение удаления БД).
    - --failfast — остановиться после первой ошибки.

    - --keepdb — не удалять тестовую БД после завершения (ускоряет повторные запуски).
    - --debug-sql — выводить SQL-запросы, выполняемые Django ORM.
- Настройка Django и запуск тестов можно посмотреть на листинге 11.

```
os.environ["DJANGO_SETTINGS_MODULE"] = "settings"
modules = list(get_test_modules())
settings.INSTALLED_APPS.extend(modules)
django.setup()

TestRunner = get_runner(settings)
test_runner = TestRunner(
   verbosity=options.verbosity,
   noinput=options.interactive,
   failfast=options.failfast,
   keepdb=options.keepdb,
   debug_sql=options.debug_sql,
)
failures = test_runner.run_tests(options.modules or modules)
sys.exit(bool(failures))
```

Листинг 11 — Код настройки Django и запуска тестов.

Скрипт указывает Django использовать settings.py из текущей директории. Добавляет найденные тестовые модули в INSTALLED_APPS (чтобы Django их обнаружил). Инициализирует Django (django.setup()). Создает экземпляр TestRunner с переданными аргументами. Запускает тесты и возвращает статус выполнения.

Помимо этого настроена конфигурация в GitHub Actions Workflow, которая автоматизирует тестирование проекта с использованием YDB при каждом изменении кода.

Этот workflow:

- Автоматически запускает тесты при:
    - Пушах в ветку master.
    - Создании или обновлении pull request`ов.
- Проверяет совместимость кода указанными версиями Python и Django.

- Обеспечивает стабильность кода, предотвращая мерж ошибок в основную ветку.
- Тесты запускаются автоматически при любых изменениях кода. на виртуальной машине развернутой на стороне Github для поддержания изоляции.
- Поддержка конкурентности, при запуске тестов позволяет экономить время и ресурсы при нескольких коммитов подряд.
- Для запуска тестов автоматически устанавливаются все нужные зависимости для проекта и разворачивается локальный YDB-сервер в Docker`e, настройки которой прописаны в docker-compose.yml файле.

3.2. Интеграционное тестирование

Для проверки работоспособности разработанного Django backend`a для YDB было проведено комплексное интеграционное тестирование. Для каждого тестового сценария были подготовлены данные и результат, который YDB возвращает для этих данных. Для каждого теста проверяется совпадает ли результат теста с заранее рассчитанным результатом.

Методология тестирования:

- Тестовая среда:
    - Использовался Docker-контейнер [18] с YDB.
    - Тестовый фреймворк: pytest с плагинами:
        - pytest-django для интеграции с Django.
        - docker для управления контейнерами.
- Тестовые сценарии:

  Целью проверки выполнения требований к реализации были протестированы следующие сценарии:
    - CRUD-операции.
    - Работа с транзакциями.
    - DDL-операции.

- Тестирование всех типов, которые поддерживаются в YDB.
- Всех методов классов backend`а.
- Базовые операции ORM.

Пример теста на проверку агрегации можно посмотреть на листинге 13.

```
aggregated_data = (
    Car.objects
    .filter(Q(make="Toyota") | Q(make="BMW"))
    .values("make", "color")
    .annotate(
        color_count=Count("id"),
        color_avg_price=Avg("price"),
        color_min_year=Min("year"),
        color_max_speed_in_stock=Max("max_speed", filter=Q(in_stock=True)),
        color_count_expensive=Count("id", filter=Q(price__gt=30000))
    )
    .order_by("make", "color")
)

results = {
    (item["make"], item["color"]): item
    for item in aggregated_data
}

toyota_black = results[("Toyota", "Black")]
self.assertEqual(toyota_black["color_count"], 5)
self.assertEqual(
    toyota_black["color_avg_price"],
    (25000 + 85000 + 32000 + 23000 + 42000) / 5
)
self.assertEqual(toyota_black["color_min_year"], 2022)
self.assertEqual(toyota_black["color_max_speed_in_stock"], 220)
self.assertEqual(toyota_black["color_count_expensive"], 3)

bmw_black = results[("BMW", "Black")]
self.assertEqual(bmw_black["color_count"], 5)
self.assertEqual(
    bmw_black["color_avg_price"],
    (65000 + 115000 + 105000 + 58000 + 67000) / 5
)
```

```
self.assertEqual(bmw_black["color_min_year"], 2023)
self.assertEqual(bmw_black["color_max_speed_in_stock"], 300)
self.assertEqual(bmw_black["color_count_expensive"], 5)
```

Листинг 13 — Пример теста на проверку агрегации.

Тестовое покрытие:

На данный момент тестовое покрытие составляет 83%. Он был рассчитан с помощью инструмента coverage.py [19], который отслеживает, какие строки кода были выполнены во время запуска тестов, а какие — нет, и предоставляет детальную статистику. Ниже можно посмотреть статистику тестового покрытия для решения. Решение успешно прошло все сценарии тестирования. Django backend работает стабильно и удовлетворяет всем требованиям. Результаты тестового покрытия можно посмотреть на листинге 14.

```
Name                                                Stmts   Miss  Cover
-------------------------------------------------------------------
...
tests/aggregates/test_aggregate.py                     72      0   100%
tests/backends/models.py                               38      6    84%
tests/backends/ydb/test_base.py                        21      0   100%
tests/backends/ydb/test_introspection.py               41      0   100%
tests/backends/ydb/test_operations.py                  78      0   100%
tests/backends/ydb/test_schema.py                      56      0   100%
tests/compiler/models.py                               33      4    88%
tests/compiler/test_delete.py                          30      0   100%
tests/compiler/test_insert.py                          24      0   100%
tests/compiler/test_update.py                          29      0   100%
tests/runtests.py                                      54      0   100%
tests/settings.py                                       3      0   100%
ydb_backend/backend/base.py                           287     52    85%
ydb_backend/backend/client.py                          18     13    28%
ydb_backend/backend/features.py                        75      2    98%
ydb_backend/backend/introspection.py                  357     30    91%
```

```
ydb_backend/backend/operations.py          367    163    55%
ydb_backend/backend/schema.py              339     91    73%
ydb_backend/backend/validation.py           40      0   100%
ydb_backend/models/sql/compiler.py         510     85    83%
...
------------------------------------------------------------
TOTAL                                     6326   1092    83%
```

Листинг 14 — Результаты тестового покрытия.

3.3. Нагрузочное тестирование

Целью нагрузочного тестирования было сравнение производительности YDB против Postgresql. Для тестирования была разработана небольшое тестовое REST [20] приложение подключенное Bookstore к одноузловому кластеру YDB в Docker. Создана таблица bookstore, для которой будут рассматриваться сценарии:

- Одиночной ставки записей.
- Множественной ставки записей.
- Извлечение записи по ключу.

Рассматриваются следующие характеристики:

- Количество запросов в секунду.
- Время ответа.

В качестве инструмента для нагрузочного тестирования был выбран инструмент Locust [21]. Он позволит сымитировать поведение пользователей, чтобы сравнить производительности YDB и Postgresql при высоких нагрузках.

Locust имеет два параметра для запуска тестов. Эти два параметра управляют нагрузкой в Locus:

- Параметр users – задает максимальное количество виртуальных пользователей, которые будут имитировать нагрузку на приложение. В качестве этого параметра берется 200 для всех тестов.

- Параметр spawn-rate – определяет скорость создания пользователей в секунду. В качестве этого параметра берется 10.

Тестирование проводились на одном и том же окружении с одними и теми же данными.

Результаты одиночной ставки записей, для Postgresql и YDB можно посмотреть на рисунках 5 и 6:



Рисунок 5 — Графики количества запросов в секунду и времени ответа в Postgresql, для одиночной вставки записей.

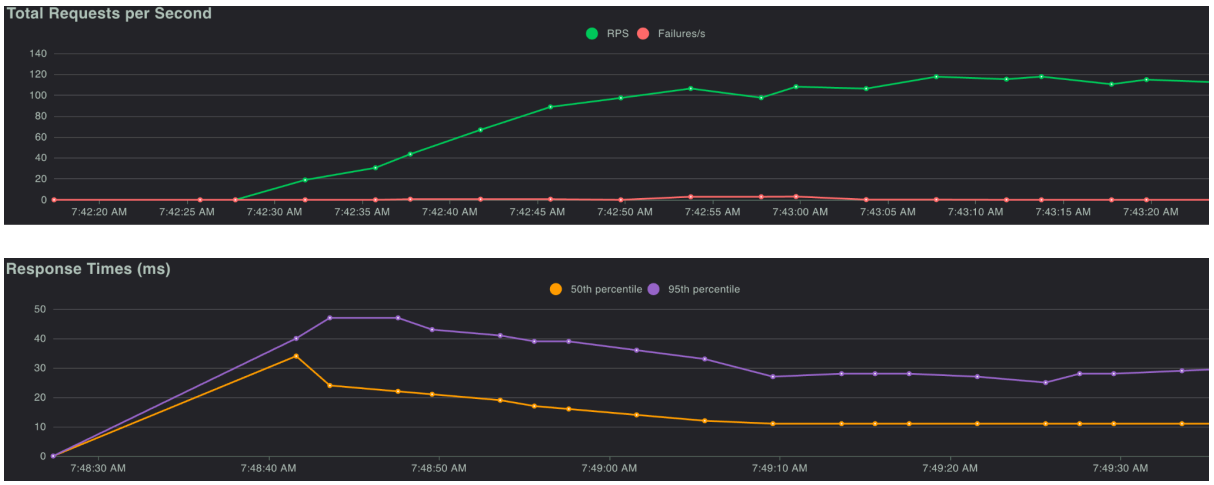
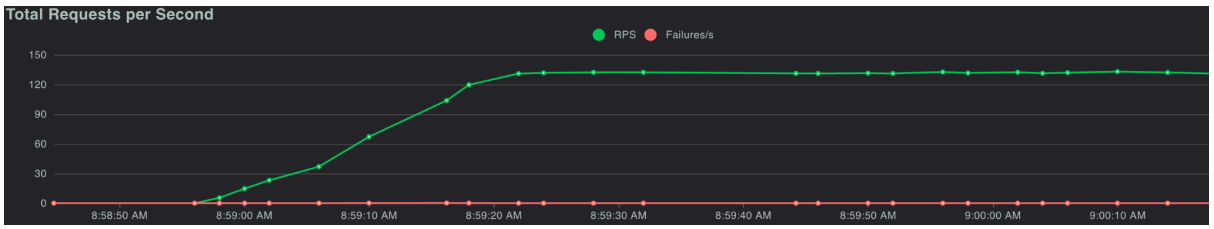


Рисунок 6 — Графики количества запросов в секунду и времени ответа в YDB, для одиночной вставки записей.

Результаты множественной ставки записей, для Postgresql и YDB можно посмотреть на рисунках 7 и 8:



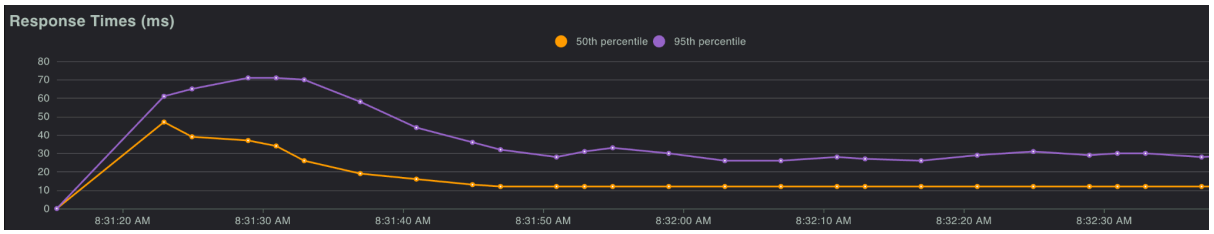
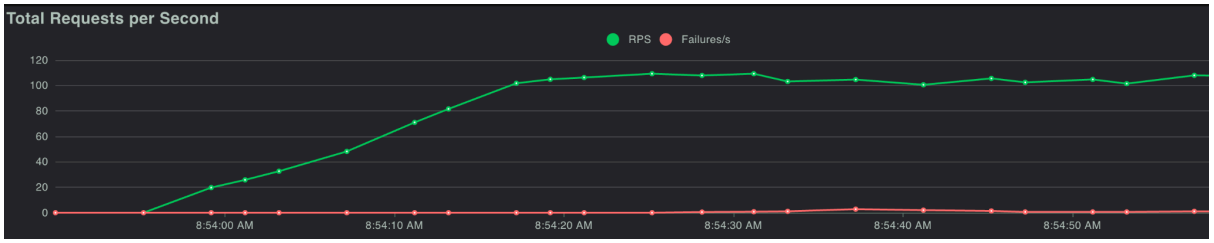


Рисунок 7 — Графики количества запросов в секунду и времени ответа в Postgresql, для множественной вставки записей.



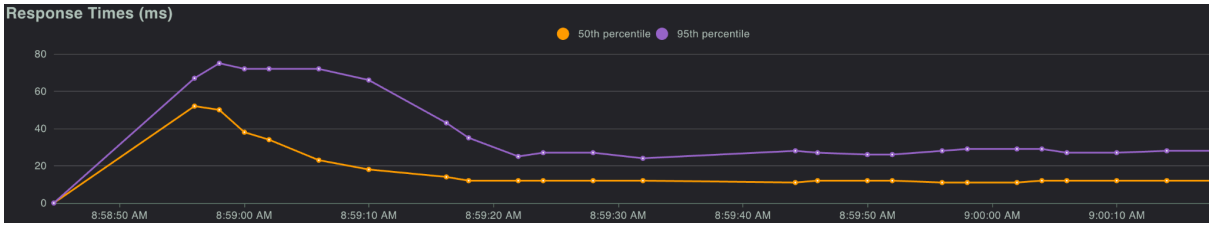
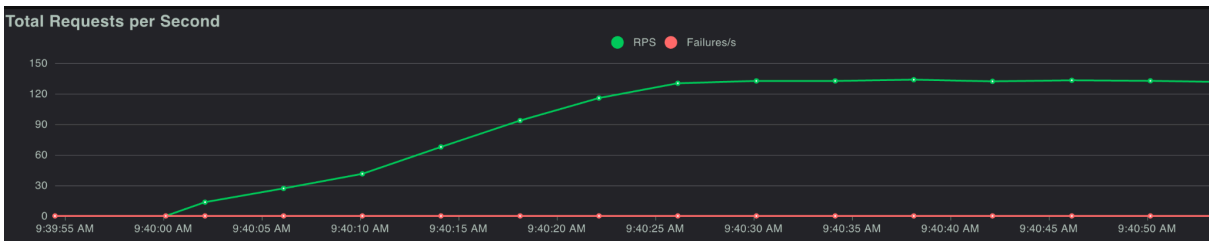


Рисунок 8 — Графики количества запросов в секунду и времени ответа в YDB, для множественной вставки записей.

Результаты извлечения записи по ключу, для Postgresql и YDB можно посмотреть на рисунках 9 и 10:



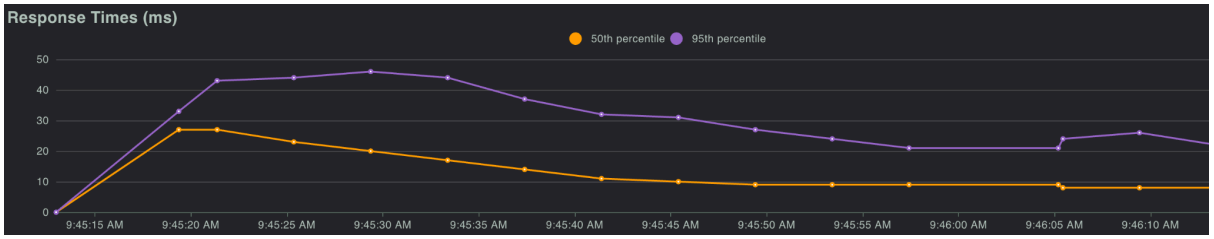


Рисунок 9 — Графики количества запросов в секунду и времени ответа в Postgresql, для извлечение записи по ключу.



Рисунок 10 — Графики количества запросов в секунду и времени ответа в YDB, для извлечение записи по ключу.

ВЫВОДЫ ПО ГЛАВЕ 3

В данной главе описаны методы тестирования и проверки корректности работы предложенного решения. Проведено интеграционное тестирование и нагрузочное тестирование. По результатам нагрузочного тестирования, приложение с использованием YDB показало допустимое снижение эффективности использования разработки по сравнению с приложением на Postgresql.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была успешно разработана и реализована интеграция модуля для работы с базой данных YDB в рамках фреймворка Django. Разработанный модуль включает все необходимые методы для полноценной разработки приложений, что позволяет использовать YDB в качестве надежного и масштабируемого решения для хранения данных.

Были достигнуты следующие результаты:

- Изучены особенности работы с YDB и её основные возможности.
- Проанализированы существующие решения для работы с YDB в Django, выявлены их преимущества и недостатки.
- Разработана архитектура интерфейса для работы с YDB в Django.
- Реализованы основные функции интерфейса, включая поддержку CRUD-операций.
- Проведено комплексное тестирование разработанного интерфейса, включая юнит-тестирование и интеграционное тестирование, нагрузочное тестирование, что подтвердило его корректность и эффективность.

Весь код разработанного решения находится в открытом официальном репозитории YDB Platform. Установить разработанное решение для работы с YDB в своих проектах, можно через репозиторий питоновских проектов PypI. Подготовлена документация с примерами использования backend`a, для поддержки новых пользователей при работе с YDB в Django. Составлен акт о внедрении, который показывает, что решение готова для использования и запуска в эксплуатацию.

Разработанное решение будет поддерживаться и дополняться в будущем и будет обеспечивать стабильную работу и полную совместимость с новыми версиями YDB.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Документация Django [Электронный ресурс]. — 2025. — URL: https://docs.djangoproject.com/en/5.2/
2. ORM [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/ORM
3. NoSQL [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/NoSQL
4. Документация YDB [Электронный ресурс]. — 2025. — URL: https://ydb.tech/docs/ru/
5. ACID-транзакций [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/ACID
6. Документация YQL [Электронный ресурс]. — 2025. — URL: https://ydb.tech/docs/ru/yql/reference/
7. Концепции YDB [Электронный ресурс]. — 2025. — URL: https://ydb.tech/docs/ru/concepts/
8. Стандарт pep 249 [Электронный ресурс]. — 2025. — URL: https://peps.python.org/pep-0249/
9. Serverless YDB [Электронный ресурс]. — 2025. — URL: https://yandex.cloud/ru/docs/ydb/concepts/serverless-and-dedicated
10. Apache License, version 2 [Электронный ресурс]. — 2025. — URL: https://www.apache.org/licenses/LICENSE-2.0
11. SQL [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/SQL
12. CRUD [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/CRUD
13. DDL [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/Data_Definition_Language

14. Репозиторий django-connector [Электронный ресурс]. — 2025. — URL: https://github.com/ecodomen/YDB-django-connector
15. Параметризованные запросы [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/Подготовленный_запрос
16. Документация Poetry [Электронный ресурс]. — 2025. — URL: https://python-poetry.org/docs/
17. Ruff [Электронный ресурс]. — 2025. — URL: https://pypi.org/project/ruff/
18. Docker [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/Docker
19. Документация Coverage [Электронный ресурс]. — 2025. — URL: https://coverage.readthedocs.io/en/latest/
20. REST [Электронный ресурс]. — 2025. — URL: https://ru.wikipedia.org/wiki/REST
21. Документация Locust [Электронный ресурс]. — 2025. — URL: https://docs.locust.io/en/stable/index.html